Thermodynamic State Vector Inventory Discrepancy Evaluator Core Helper: Enforcing Conservation Laws in the Web of Life Framework (Sprint 067)

Lead Scientific Communications & Academic Outreach Agent
Web of Life

Sprint 067 – Repository: `https://github.com/pascalranoroarijaona/WebOfLife`

Abstract

In biogeochemical modeling and industrial process simulation within the *Web of Life* framework, monad state transitions represent mass-energy transformations across living and abiotic compartments. To enforce rigorous adherence to physical conservation laws (First Law of Thermodynamics / Conservation of Mass) and state boundary constraints (Second Law entropy and dissipation limits), continuous state validation is required. Sprint 067 introduces the Thermodynamic State Vector Inventory Discrepancy Evaluator Core Helper (`src/thermodynamics/state_validator.ts`), providing isolated, deterministic mathematical comparison routines for checking absolute elemental stock discrepancies against defined individual elemental tolerances.

1 Introduction and Thermodynamic Foundation

Ecosystems and simulated biophysical networks operate as open thermodynamic systems maintaining far-from-equilibrium steady states through continuous energy flux and material cycling. Within the *Web of Life* computational architecture, monad state vectors ($\vec{S}$) encapsulate elemental stocks (C, N, P, H_2O, etc.).

To maintain physical plausibility and track exergy dissipation, state transitions must strictly obey:

1. **First Law of Thermodynamics (Conservation of Matter):** Elemental inventories must not spontaneously appear or vanish outside accepted floating-point numerical tolerances.
2. **Second Law of Thermodynamics (Dissipative Bounds):** Discrepancies exceeding allowable thresholds flag entropy accounting errors, non-conservative monad state transformations, or boundary leakage.

2 Mathematical Formalization

Let an arbitrary thermodynamic state vector $\vec{S}$ be defined as a mapping from elemental stock keys $k \in K$ to scalar quantities $q_k \in \mathbb{R}$ representing moles, mass (kg), or energy equivalents (Joules).

Given an actual state vector $\vec{S}_{\text{actual}}$ and an expected state vector $\vec{S}_{\text{expected}}$, the absolute inventory discrepancy Δ_k for any stock k is computed as:

$$\Delta_k = |q_{k,\text{actual}} - q_{k,\text{expected}}| \tag{1}$$

For each stock k, let τ_k be the allowable tolerance defined either by a custom per-element mapping or a global default tolerance τ_{global}:

$$\tau_k = \begin{cases} \text{customTolerances}[k] & \text{if } k \in \text{customTolerances} \\ \tau_{\text{global}} & \text{otherwise} \end{cases} \tag{2}$$

The validation predicate $\mathcal{P}_k$ for stock k and the global validation result IsValid are defined as:

$$\mathcal{P}_k = (\Delta_k \leq \tau_k), \quad \text{IsValid} = \bigwedge_{k \in K} \mathcal{P}_k \tag{3}$$

3 Implementation Specification

The validation helper class is implemented in TypeScript within `src/thermodynamics/state_validator.ts`:

Listing 1: StateValidator Implementation

```
import { StateVector } from './state_vector';
import { ValidationResult } from './types';

blueexport blueclass StateValidator {
  blueconstructor(blueprivate globalTolerance: number = 1e-6) {}

  bluepublic evaluateDiscrepancy(
    actual: StateVector,
    expected: StateVector,
    customTolerances?: Record<string, number>
  ): ValidationResult {
    blueconst differences: Record<string, number> = {};
    blueconst violations: Record<string, string> = {};
    let isValid = true;

    blueconst actualStocks = actual.getStocks();
    blueconst expectedStocks = expected.getStocks();
    blueconst keys = bluenew blueSet([...blueObject.keys(actualStocks), ...blueObject.k

    for (blueconst key of keys) {
      blueconst actVal = actualStocks[key] ?? 0;
      blueconst expVal = expectedStocks[key] ?? 0;
      blueconst diff = Math.abs(actVal - expVal);
      differences[key] = diff;

      blueconst tolerance = customTolerances?.[key] ?? this.globalTolerance;
      blueif (diff > tolerance) {
        isValid = false;
        violations[key] = `Discrepancy ${diff} exceeds tolerance ${tolerance}`;
      }
    }

    bluereturn {
      isValid,
      differences,
      violations
    };
  }
}
```

4 Verification Vectors

Table 1 summarizes the verification test cases validating the discrepancy evaluator core helper.

Test ID	**Actual Vector** ($q_{\mathbf{actual}}$)	**Expected Vector** ($q_{\mathbf{expected}}$)	**Tolerances** (τ)	**Expected Out**
TC-01	{ C: 100.0, H2O: 500.0 }	{ C: 100.0, H2O: 500.0 }	Default (10^{-6})	`isValid: tru`
TC-02	{ C: 100.000005, N: 10.0 }	{ C: 100.0, N: 10.0 }	Default (10^{-6})	`isValid: fal`
TC-03	{ C: 100.005, N: 10.0 }	{ C: 100.0, N: 10.0 }	{ C: 0.01 }	`isValid: tru`

Table 1: Verification vectors for Sprint 067 state validation.

5 Conclusion

Sprint 067 successfully delivers the `StateValidator` core helper, establishing a robust computational guardrail for thermodynamic state transitions within the *Web of Life* repository (`https://github.com/pascalranoroarijaona/WebOfLife`). Future work will integrate real-time entropy production monitoring alongside mass conservation checks.